PA-1

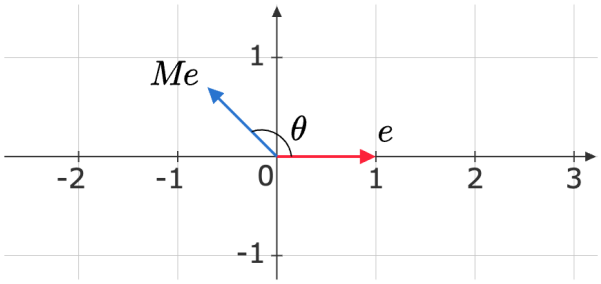
Instructions

- This assignment will count towards bonus marks. This is PA-1. One more assignment will be released later. These two together can fetch you at most 5/100 in the total score.
- This assignment is not going to affect your eligibility to write the end-term. That is only dependent on the graded assignments.
- You have to submit a report along with the code.
- Use this form to submit your report and code.
- The deadline is 20th July, 2025.

JL Lemma

Question-1: Visualizing random projections

Construct $m = 10,000$ random matrices, $M_1, \cdots, M_m$. For each random matrix, M_i, record the angle (in degrees) between the vectors e and $M_i e$. Plot the histogram of these angles and write down your observation.



$$M = \begin{bmatrix} m_{11} & m_{12} \\ m_{21} & m_{22} \end{bmatrix}$$

$$m_{ij} \sim \mathcal{N}(0, 1)$$

If you are wondering why we aren't dividing by $\sqrt{2}$ here, we are only interested in the angles and not the distanaces for this question.

Question-2: JL From Scratch

Consider a dataset X of shape $d \times n$, where $d = 10,000$ and $n = 5,000$ that can be obtained by running the code-block given below. Construct an approximate isometry, $f : \mathbb{R}^d \to \mathbb{R}^k$, and project the dataset to the lower dimensional space, $\mathbb{R}^k$, such that for all pairs of points (x_i, x_j) in the dataset:

$$(1 - \epsilon)||x_i - x_j||^2 \leqslant ||f(x_i) - f(x_j)||^2 \leqslant (1 + \epsilon)||x_i - x_j||^2$$

where $\epsilon = 0.2$. Ensure that k is significantly less than d.

- What value of k did you obtain?
- How many random matrices did you have to generate?

Run the following cell to get the data.

```
# DATA CELL #
# DO NOT EDIT THIS CELL #
import numpy as np
def get_data(d, n):
    rng = np.random.default_rng(seed = 1234)
    X = rng.uniform(
    size=(d, n)
    )
    return X
d, n = 10_000, 5_000
X = get_data(d, n)
```

Question-3: Effect of k on pair-wise distances

Consider a dataset X of shape (d, n) with $d = 2000$ and $n = 1000$ that can be obtained by running the code-block given below. Construct random projection matrices – as specifed by the JL Lemma – that project to $\mathbb{R}^k$ for the following values of k:

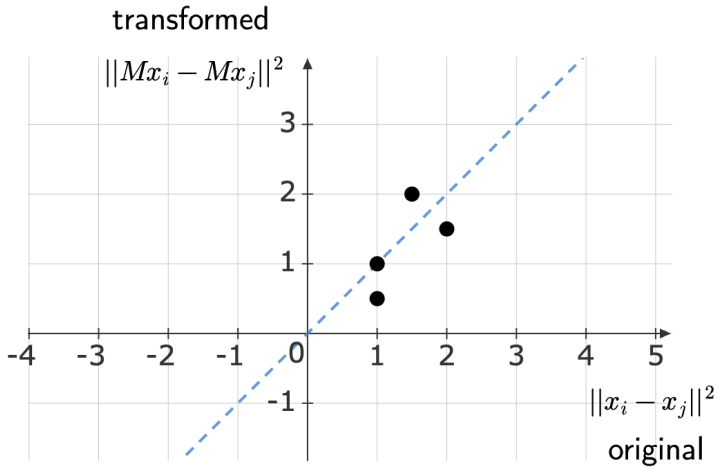
- $k = 100$
- $k = 500$
- $k = 1500$

For each of these projection matrices, plot a 2D heatmap of the pairwise distances of the points before and after the transformation. Use `red` to indicate a higher density and blue to indicate lower density of points in a region. Compare the three heatmaps and record your observations.

Get the data by running this:

```
# DATA CELL
# DO NOT EDIT THIS CELL
def get_sparse_data(d, n):
    rng = np.random.default_rng(seed = 1234)
    return rng.choice(
        [0, 1],
        p=[0.8, 0.2],
        size=(d, n)
    )
d, n = 2_000, 1000
X = get_sparse_data(d, n)
```

For your reference, the following is a scatter plot of distances for four pairs.



Question-4: Linear Separability

Run the code given below to initialize a linearly separable dataset (X, y) for a binary classification problem, where the shape of X is (d, n) with $d = 2000, n = 1000$ and y is the label vector of shape $(n,)$.

- **Verify**: Fit a perceptron to this dataset and verify its linear separability.
- **Experiment**: Consturct a random projection matrix – as specified by the JL Lemma – that projects this dataset to $\mathbb{R}^k$. Fit a perceptron on this transformed dataset and measure the accuracy of the resulting model. This consititutes one experiment.

- **Cluster**: Run this experiment for the following values of k: $\{100, 150, \cdots, 1000\}$. This set of experiments is one cluster. Run the cluster 50 times and average the accuracies across the 50 runs.
- **Plot**: Plot the accuracies as a function of k and determine the smallest value of k at which the perceptron achieves a 99% accuracy. Record your observations.

Use `sklearn.linear_model.Perceptron` in its default configuration for fitting all perceptrons in your experiments.

```
# DATA CELL
# DO NOT EDIT THIS CELL
rng = np.random.default_rng(seed=1035)
d = 2_000
n = 1_000

w = rng.uniform(-2, 2, d)
X = rng.normal(size=(n, d))
y = np.where(X @ w >= 0, 1, 0)
```

LSH

Question 1: Preparing the Dataset

Obtain a *large* dataset $\mathcal{D}$ with m rows and n columns, satisfying $mn > 10000$, $m > 1000$, and $n > 10$ (the larger the dataset, the better). You may choose any suitable dataset available online (a few examples are listed below), or you can create your own dataset using images, documents, audio files, etc. In the latter case, you may need to process these files (e.g., by generating embeddings) to construct a $m \times n$ data matrix. Note that the dataset does not need to be labeled. Clearly explain the steps you follow to create or collect this dataset.

- GTZAN – Music Genre Classification
- Fashion MNIST
- Audio MNIST

Question 2: Implementing LSH

Write Python code to implement Euclidean LSH. Organize your code into small, logical blocks, and provide a brief explanation for each block.

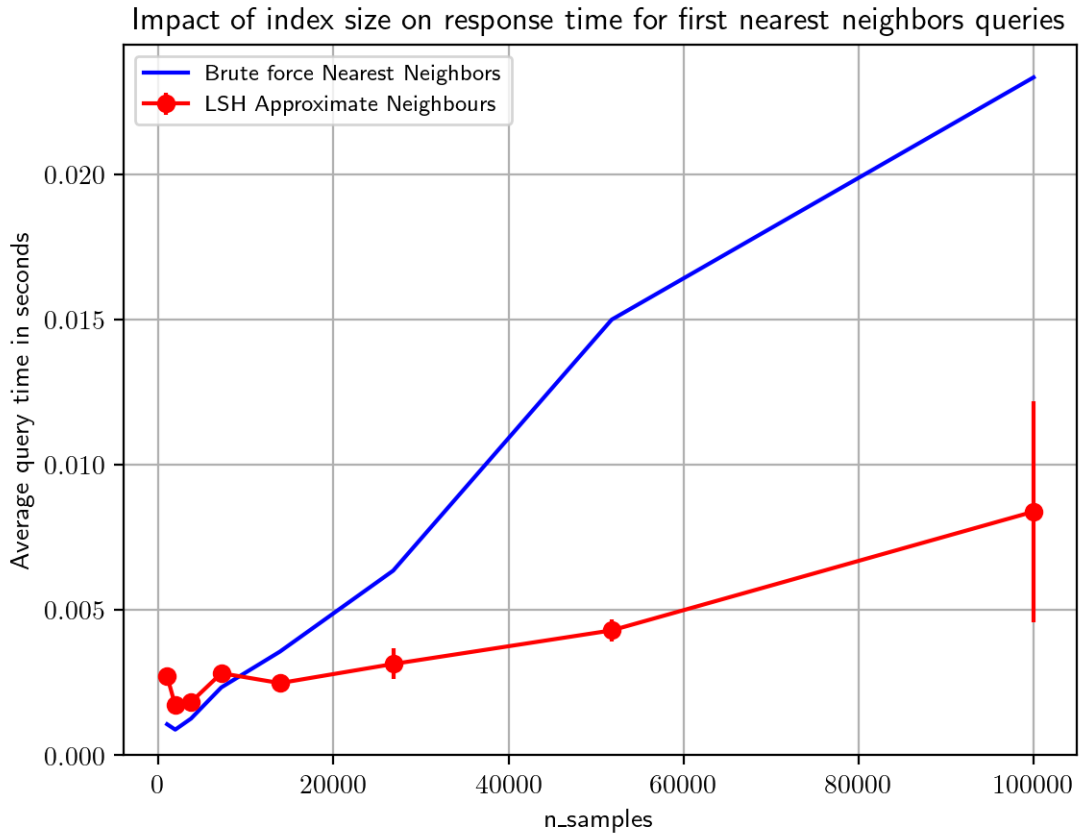
Question 3: Choosing LSH Parameters

Apply your LSH implementation to the dataset and test whether it functions as expected. You will need to experiment with the parameters w, L, and k to achieve desirable performance. Describe your approach for choosing suitable values for these parameters, and discuss how each parameter influences the behavior and results of LSH.

Question 4: Comparing LSH and Brute Force Nearest Neighbors

Evaluate the performance of LSH nearest neighbors compared to brute force nearest neighbors as the dataset size increases. Select at least six values $n_i < m$, and for each n_i, create a dataset $\mathcal{D}_i$ by randomly selecting n_i rows from $\mathcal{D}$. For each $\mathcal{D}_i$, measure the inference time t for both LSH and brute force nearest neighbors, and plot t against dataset size. Since LSH performance is probabilistic, estimate t by sampling multiple times and include error bars of size $\pm\sigma$ in the plot.

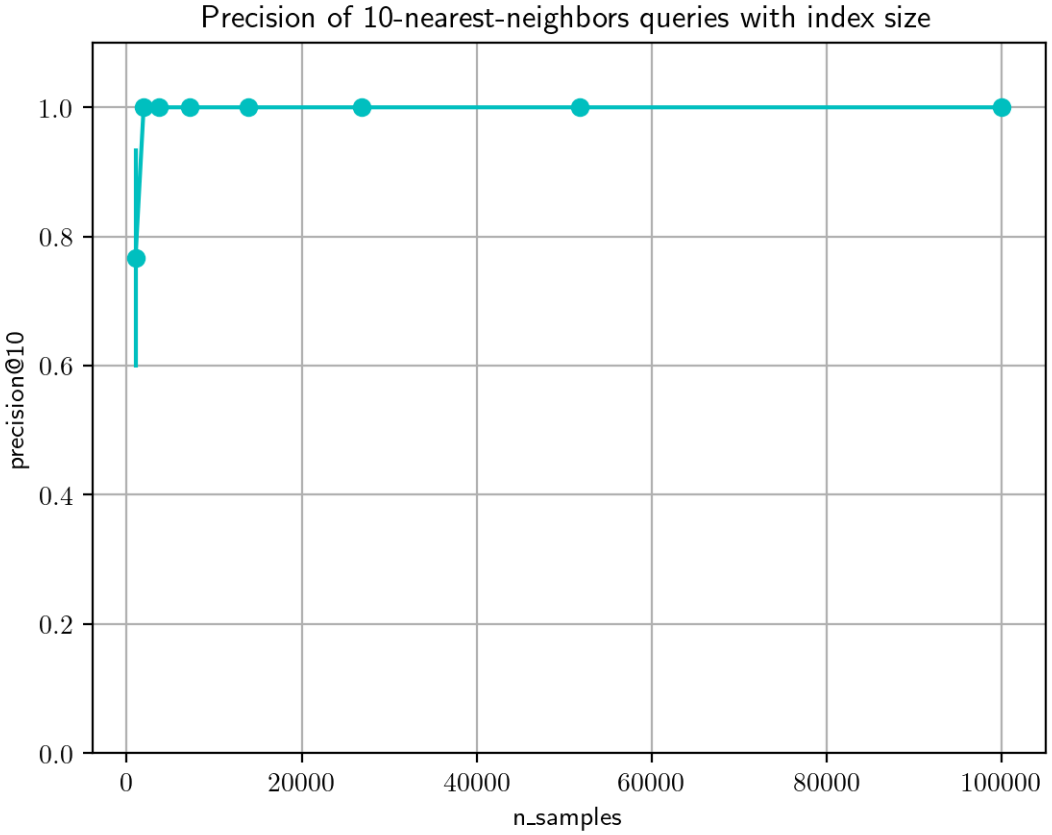
Example plot:



Question 5: Evaluating LSH Precision

While brute force nearest neighbors return the exact top K neighbors, LSH is approximate. To assess the accuracy of your LSH implementation, compute, for each $\mathcal{D}_i$, the proportion of the top 10 neighbors returned by LSH that are also found by brute force. Plot this precision metric for varying dataset sizes.

Example plot:



Randomized SVD

Question-1: Power-iteration

Generate a random matrix A of shape $(100, 100)$ with each entry drawn independently from a standard normal distribution: $A_{ij} \sim \mathcal{N}(0, 1)$. Compute the SVD of the matrices $(AA^T)^q A$ for $q \in \{0, 1, 2, 3, 4, 5\}$. Plot the six sets of singular values on a common plot with logscale on the y-axis. Describe what you observe and provide an explanation for the same.

Question-2: PCA

Download a dataset of human faces from the web that has at least 5000 images.

- Convert all images to grayscale, resize them to 100×100 and scale them so that the pixel values are in the range $[0, 1]$.
- Center the dataset.
- Run PCA on the centered dataset using randomized SVD as the solver retaining the top 20 components. You can refer to scikit-learn's PCA function.
- Plot σ_i versus i, where σ_i is the i^{th} singular value, for $1 \leq i \leq 20$.
- Visualize the first 20 singular vectors as images.

Some datasets you can use:

- Human faces, Kaggle
- Caltech

Note that you don't need to implement randomized SVD from scratch. Use the implementation provided by scikit-learn.